Universidad de Ingeniería y Tecnología

Departamento de Ingeniería Mecatrónica

Fundamentos de Robótica

Profesores: Oscar E. Ramos, Alexander López

Laboratorio 3

Modelos de Robots usando URDF

Lima - Perú

2026-1

Laboratorio 3
Modelos de Robots usando URDF

Objetivos

- Conocer el formato URDF y algunos de sus parámetros más importantes.
- Modelar un robot utilizando el formato URDF y visualizarlo en RViz.

Equipo y Materiales

Descripción	Cantidad
Computadora con Internet	1

Indicaciones del Laboratorio

1. Todos los laboratorios son evaluados, por tanto se debe leer la guía del laboratorio antes de asistir a cada laboratorio.
2. El laboratorio comienza a más tardar 5 minutos después de la hora de inicio.
3. A los 5 minutos de iniciar el laboratorio se empieza con la prueba de entrada, la cual se debe desarrollar de manera personal. Esta prueba de entrada cuenta como parte de la nota del laboratorio, la cual evalúa que el alumn@ haya leído la guía de laboratorio. La prueba tiene una duración de 3 o 4 minutos, dependiendo la cantidad de preguntas.
4. Los alumnos que lleguen 20 minutos después de la hora inicio del laboratorio, son libres de asistir, pero no tendrán nota en dicho laboratorio.
5. El laboratorio se desarrolla en pareja de 2 personas (solo será de 3 personas en caso falte un alumn@ que no tenga pareja).
6. La prueba de salida será evaluada por medio de un cuestionario colgado en canvas, el cuestionario es llenado por cada uno de los integrantes del grupo, se puede realizar la prueba de manera grupal, pero cada alumno lo debe hacer desde su propio canvas.
7. La parte práctica será revisada por el profesor durante el laboratorio, es posible que se haga preguntas de los códigos desarrollados.
8. Está prohibido copiar, compartir información o conversar con otro grupo. Los alumnos involucrados serán presentados al comité de ética de la universidad.

Resumen del laboratorio

Se presenta un resumen de los puntos a tratar en el siguiente laboratorio.

1. Prueba de entrada (4 puntos)
2. Descargar los paquetes para este laboratorio.
3. Diseño de robots usando URDF.
4. Visualización del Modelo con RViz.
5. Xacro para la Creación de Modelos URDF.
6. Evaluación en Canvas (11 puntos)
7. Práctica (5 puntos).

1. Paquetes del laboratorio

Para trabajar en este laboratorio, es necesario descargar el repositorio del curso en el espacio de trabajo "lab_ws":

```
$ cd lab_ws/src
$ git clone https://github.com/utecrobotics/fundrobotica-20261 frlabs
$ cd ..
$ colcon build
```

En caso que el espacio de trabajo "lab_ws" haya sido borrado, el alumn@ debe volver a crearlo y hacer que el sistema lo reconozca como un espacio de trabajo de ROS2. Además, se deben instalar los siguientes paquetes:

```
$ sudo apt-get install ros-humble-control* ros-humble-joint* ros-humble-ros2-control*
$ sudo apt-get install ros-humble-gazebo* ros-humble-forward* ros-humble-xacro
$ sudo apt-get install ros-humble-hardware-interface ros-humble-gz*
```

2. Diseño de robots usando URDF

En está sección se enseñará a cómo diseñar un robot en formato urdf, el diseño terminado podrá ser visualizado (usando Rviz) o simulado (usando Gazebo o Ignition). En el paquete del laboratorio se han programado ejemplos sencillos, los cuales irán incrementando su complejidad, hasta llegar a un diseño final apto para simulación.

El formato de descripción de robótica unificada (urdf) es un tipo de archivo de lenguaje de marcado extensible (XML), con el cual se puede realizar la descripción física de un robot. El alumn@ debe inspeccionar el contenido de los archivos dentro de la carpeta urdf en el paquete "lab3", el cual contiene las etiquetas robot, link y joint, explicados en el apéndice A. Durante el

laboratorio se realizará un explicación complementaria acerca de los modelos urdf presentes en el paquete “lab3”.

Dentro del paquete, en la carpeta urdf, abrir el archivo llamado “robot.urdf”. Las etiquetas de tipo <visual> describen la apariencia del eslabón, la cuál se mostrará en la visualización y simulación del robot. Dentro de las etiquetas <visual> se define la geometría del robot como cilindros (cylinder), cubos (box), esferas (sphere) o mallas 3D (mesh). **Es importante mencionar que todas las medidas usadas en ROS2 usan las unidades del sistema internacional de unidades, notar que las longitudes siempre se encuentran en metros y los ángulos están en radianes**. Además, los eslabones se pueden personalizar cambiando su color (o textura) y forma.

Cuando se desarrolla un nuevo diseño de robot usando el formato urdf, es posible verificar si dicho modelo está correctamente programado o si presenta algún problema de sintaxis usando el comando “check_urdf”. Para verificar el modelo “robot.urdf” ejecutar los siguientes comandos desde un terminal:

```
$ cd # ubicarse en la raíz
$ cd lab_ws/src/frlabs/lab3/urdf
$ check_urdf robot.urdf
```

Este programa realiza el análisis del modelo urdf y muestra si es correcto con el mensaje “OK”, así como una estructura simplificada de los eslabones del robot, cuando ningún problema es detectado; de lo contrario, muestra un error. Para el caso de robots más complejos resulta útil visualizar de manera gráfica la estructura de los eslabones del robot y sus articulaciones. Para ello, se puede utilizar el comando llamado “urdf_to_graphiz” de la siguiente forma:

```
$ cd # hacer esto solo si ha salido de la carpeta urdf
$ cd lab_ws/src/frlabs/lab3/urdf # hacer esto solo si ha salido de la carpeta urdf
$ urdf_to_graphiz robot.urdf
```

Este comando genera un archivo pdf con el nombre del robot (que puede no ser igual al nombre del archivo urdf, como en este caso). El pdf generado puede ser abierto con cualquier programa que permita la lectura de pdfs (como evince en Ubuntu).

3. Visualización del Modelo con RViz

Luego de tener el modelo del robot en URDF, diseñado u obtenido a partir del fabricante, este puede ser visualizado en RViz. Dentro de la carpeta launch del paquete “lab3” existe un archivo llamado “robot.launch” con el siguiente contenido:

robot.launch

```
<?xml version="1.0"?>
<launch>
  <let name="urdf_path" value="$(find-pkg-share lab3)/urdf/robot.urdf" />
  <node pkg="joint_state_publisher_gui" exec="joint_state_publisher_gui" />
  <node pkg="robot_state_publisher" exec="robot_state_publisher">
    <param name="robot_description" value="$(command 'xacro $(var urdf_path)')"/>
  </node>
  <node pkg="rviz2" exec="rviz2" name="rviz2" args="-d $(find-pkg-share lab3)/config/robot.rviz" />
</launch>
```

Este archivo “robot.launch” corre los siguientes nodos y define ciertos parámetros:

- robot_description: Es un parámetro que almacena el modelo URDF del robot. En el archivo launch se le debe indicar el camino al modelo del robot (en este caso al modelo xacro, que genera el modelo urdf).
- joint_state_publisher_gui: Crea sliders para cada articulación y publica los valores articulares en el tópico “joint_state”.
- robot_state_publisher: Lee los valores angulares del tópico joint state y los convierte en transformaciones de posición y orientación para cada cuerpo rígido del robot en formato tf.
- rviz2: Lanza RViz2 con el nodo RViz2, y pertenece al paquete del mismo nombre. Además se le especifica el argumento, al cuál se le especifica la dirección exacta de un archivo que contiene la información relacionada con la ventana de RViz2 de tal modo que la siguiente vez mantenga las mismas propiedades que tenía en la anterior sesión.

Este archivo permite visualizar el modelo del robot usando el comando “ros2 launch”:

```
$ ros2 launch lab3 robot.launch
```

Por defecto, no existe ningún elemento de tipo robot agregado en RViz2. Para poder visualizar el robot se debe seguir los siguientes pasos:

1. Especificar el sistema que se utilizará como sistema de referencia: en la lista desplegable correspondiente a “Fixed_Frame” escoger “base_link”.
2. Agregar un modelo de robot: Add → RobotModel → OK.
3. Seleccionar el “Description topic” la opción “/robot_description”

Verificar usando los 2 sliders que los límites articulares sean los especificados en el modelo URDF y verificar que los nombres de las articulaciones (en los sliders) coincidan con los especificados en el modelo. Luego, para guardar la configuración actual hacer clic en: File →

Save Config. Si se desea visualizar los sistemas de referencia asociados con cada uno de los eslabones, se puede añadir un elemento de tipo TF: Add → TF → OK.

4. Xacro para la Creación de Modelos URDF

El formato URDF por defecto solamente permite especificar valores numéricos constantes para longitudes, masas, inercias, etc. Así, si se quiere por ejemplo, ingresar la misma longitud para dos eslabones, es necesario especificar el mismo número dos veces, y si uno cambia, se debe manualmente cambiar el otro (en caso que se desee siempre la misma longitud). De igual manera, si se desea especificar la mitad de un valor, es necesario ingresar dicho valor de forma manual, como una constante. Esto reduce la flexibilidad de URDF para trabajar con modelos complejos de robots. **Para solucionar estos problemas, se creó el formato xacro, el cual permite crear variables (llamadas propiedades), macros que pueden ser reutilizadas, así como algunas operaciones matemáticas**. La siguiente es una breve descripción de estos elementos añadidos en xacro:

- **Propiedades**: Son variables asociadas a un valor y se definen de la siguiente manera: . Luego de definida esta propiedad, para recuperar su valor se utiliza: ${nombre var}.
- **Operaciones Matemáticas**: Se puede realizar algunas operaciones matemáticas elementales; por ejemplo: ${nombre_var+0.5}.
- **Macros**: Se comportan como funciones, y admiten parámetros de referencia como entradas. Por ejemplo, la siguiente es una macro que toma como parámetro de entrada la masa (mass) y crea una etiqueta de valores inerciales con la masa ingresada y con un tensor de inercia predefinido.

```
<xacro:macro name="inertial_matrix" params="mass">
<inertial>
  <mass value="${mass}" />
  <inertia ixx="0.5" ixy="0.0" ixz="0.0" iyy="0.5" iyz="0.0" izz="0.5" />
</inertial>
</xacro:macro>
```

Una vez declarada la macro, esta puede ser utilizada desde el archivo donde se define el modelo del robot, que podría ser el mismo archivo donde se definió la macro, u otro archivo. En este ejemplo, la macro puede ser instanciada como: <xacro:inertial_matrix mass="1" />

Luego, a partir del formato xacro es posible obtener un archivo con formato urdf como se revisará más adelante.

4.1. Conversión de xacro a URDF

Los archivos con formato de tipo xacro pueden ser fácilmente convertidos a formato URDF utilizando el script llamado xacro.py. Por ejemplo, para convertir a urdf el archivo xacro "robotx.xacro", se puede utilizar la siguiente secuencia de comandos:

```
$ cd # ubicarse en la raíz
$ cd lab_ws/src/frlabs/lab3/urdf
$ ros2 run xacro xacro robotx.urdf.xacro > robotx.urdf
```

4.2. Visualización en RViz

Dentro de la carpeta launch se encuentra el archivo "robotx.launch" con el contenido mostrado a continuación. Este archivo será utilizado para visualizar el modelo.

robotx.launch

```
<?xml version="1.0"?>
<launch>
  <let name="urdf_path" value="$(find-pkg-share lab3)/urdf/robotx.urdf.xacro" />
  <node pkg="joint_state_publisher_gui" exec="joint_state_publisher_gui" />
  <node pkg="robot_state_publisher" exec="robot_state_publisher">
    <param name="robot_description" value="$(command 'xacro $(var urdf_path)')"/>
  </node>
  <node pkg="rviz2" exec="rviz2" name="rviz2" args="-d $(find-pkg-share lab3)/config/robotx.rviz" />
</launch>
```

Ejecutar el archivo creado usando "ros2 launch" como en el caso anterior pero, obviamente, usando "robotx.launch" en lugar de "robot.launch".

4.3. Modelo en Gazebo

Mientras que RViz se utiliza solo para visualizar el modelo del robot, Gazebo es un programa que permite la simulación dinámica del robot. Debido a ello, para que un modelo pueda ser simulado usando Gazebo es necesario añadir, además de la información puramente geométrica (usada en RViz), información relacionada con las propiedades dinámicas de cada elemento. Algunos de los elementos que es necesario a˜nadir para la simulación dinámica se describen en el apéndice B.

4.3.1 Simulación del Modelo en Gazebo

El archivo "urdf/robot_gazebo.urdf" contiene las etiquetas mencionadas en el apéndice B, además de algunas otras etiquetas adicionales necesarias para el funcionamiento adecuado de

Gazebo. Además, el archivo "launch/robot_gazebo.launch" contiene el archivo de lanzamiento necesario para ejecutar gazebo y visualizar el modelo urdf dentro de este entorno de simulación. Visualizar el modelo en Gazebo utilizando el siguiente comando:

$ ros2 launch lab3 robot_gazebo.launch

Visualizar el modelo en Ignition utilizando el siguiente comando:

$ ros2 launch lab3 robot_gz.launch

Notar que el modelo podría tender a moverse hacia algún lado debido a los parámetros dinámicos asignados a cada elemento.

4.3.2 Movimiento del Robot en Gazebo

Para mover cada articulación se añade un controlador de ROS2, el cual está indicado en la etiqueta transmission. Este controlador consiste principalmente de un mecanismo realimentado (normalmente un bucle PID), el cual recibe una referencia y controla la salida usando la realimentación de los actuadores. El controlador de ROS2 se comunica con la interfaz de hardware. **La función principal de la interfaz de hardware es actuar como sistema mediador entre los controladores de ROS2 y el hardware real o el simulador**.

De este modo, el mismo controlador y el mismo código puede accionar un robot simulado o un robot real. Para realizar la interfaz entre los controladores se define un archivo de tipo yaml (que es un estándar de archivos con un formato específico). Dentro de la carpeta config, se encuentra el archivo llamado "pid_controllers.yaml".

En este archivo de extensión yaml se especifica el tipo de controlador que se utilizará para cada articulación, así como los valores proporcional, integral y derivativo de cada controlador de bajo nivel.

pid_controllers.yaml

```
controller_manager:
  ros__parameters:
    update_rate: 30

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    pid_controller:
      type: pid_controller/PidController
```

```
pid_controller:
  ros__parameters:
    dof_names:
      - pan_joint
      - tilt_joint
    command_interface: position
    reference_and_state_interfaces: ["position"]
    gains:
      pan_joint: {p: 0.2, i: 0.01, d: 0.0001, i_clamp_max: 5.0, i_clamp_min: -5.0, feedforward_gain: 1.0}
      tilt_joint: {p: 0.2, i: 0.01, d: 0.0001, i_clamp_max: 5.0, i_clamp_min: -5.0, feedforward_gain: 1.0}
```

Para realizar la simulación dinámica del modelo creado en Gazebo, ejecutar el archivo "robot_gazebo_control.launch" que se encuentra en la carpeta launch. Se debe observar que aparece el robot en Gazebo. Inspeccionar los tópicos existentes con:

```
$ ros2 topic list
```

Luego, se puede mover las articulaciones con posiciones arbitrarias. Para realizar una prueba de esto, utilizar el comando "ros2 topic pub", el cual realiza la publicación de un mensaje en un tópico determinado. En este caso, el tópico será el controlador de la primera articulación:

```
$ ros2 topic pub /pid_controller/reference control_msgs/msg/MultiDOFCommand "{dof_names: ['pan_joint','tilt_joint'], values: [0.5,0.5]}"
```

También se puede operar el robot en Ignition, para simular el robot con controlador "forward_position_controller", puede cancelar todos los terminales anteriores y abrir un nuevo terminal y correr el siguiente comando:

```
$ ros2 launch lab3 robot_gz_control.launch
```

Para mover el robot puede utilizar el siguiente comando en un nuevo terminal:

```
$ ros2 topic pub /forward_position_controller/commands std_msgs/msg/Float64MultiArray "{data: [0.5,0.5]}"
```

De igual manera, se puede enviar un comando articular a la otra articulación. Además, se puede utilizar un programa de Python para enviar el valor deseado a cada articulación. Se puede probar el envío desde Python utilizando el archivo "send_joints" que se encuentra en la carpeta "lab3/lab3". Ejecutar el programa (mientras esté corriendo la simulación de Gazebo) con el comando:

```
$ ros2 run lab3 send_joints
```

En caso de trabajar con ignition o "gz" en la computadora en la nube, se debe descargar el siguiente paquete:

```
$ cd
$ cd lab_ws/src
$ git clone -b humble https://github.com/ros-controls/gz_ros2_control.git
$ cd ..
$ colcon build
```

En caso de no compilar los paquetes y resulte un error, ejecutar varias veces el comando "colcon build", entre unas 3 a 6 veces hasta que compile correctamente todos los paquetes. Luego podrás correr la simulación en ignition con el controlador y este funcionará correctamente.

Apéndice

A - URDF

A.1. Introducción

ROS2 (Robot Operating System) provee paquetes que pueden ser utilizados tanto para construir modelos 3D de robots como para comunicarse con estos modelos a través de señales que comandan su movimiento. **El formato URDF (Unified Robot Description Format) permite definir modelos de robots a través de la especificación de cada eslabón, con sus características físicas (como masa, tensor de inercia, longitudes, etc.), y de cada articulación**. Además, se puede definir actuadores, diversos sensores, y objetos para un entorno de trabajo. Sin embargo, actualmente el formato URDF está restringido a robots que poseen eslabones rígidos.

Los modelos del formato URDF siguen la convención de los archivos de tipo XML (eXtensible Markup Language), y como tal, están compuestos por etiquetas especiales de XML que pueden ser leídas para la extracción de la información que contienen. A la lectura del archivo para extraer la información importante del modelo se denomina parsing y al programa o función que lo realiza, "parser". **ROS2 provee herramientas para realizar la extracción de información de un modelo URDF, pero estas herramientas hacen uso principalmente de C++**.

Algunos paquetes y herramientas útiles para la interacción con modelos URDF son los siguientes:

- joint_state_publisher: Este paquete contiene un nodo del mismo nombre que lee la descripción del modelo del robot (en formato URDF), encuentra todas las articulaciones (joints) y publica los valores articulares (q) para todas las articulaciones movibles usando "sliders".
- robot_state_publisher: Este paquete lee el estado de las articulaciones del robot (q) y publica las posiciones y orientaciones 3D de cada eslabón (xi) usando la cinemática directa obtenida a partir del modelo URDF. La posición 3D del robot se publica como una transformación (tf) que define las relaciones entre los sistemas de coordenadas.
- xacro: Significa Xml mACROs y es un formato que permite utilizar variables y otros add-ons para la generación de modelos complejos en formato URDF. De esta forma los modelos pueden ser más fáciles de entender y más mantenibles.

A.2. Modelado de un robot con URDF

Un modelo URDF puede representar la descripción cinemática y dinámica de un robot, su representación visual, y el modelo de colisión. Al igual que cualquier archivo xml está basado en etiquetas. Las etiquetas comúnmente utilizadas en URDF son las siguientes.

1. link: Cada etiqueta de este tipo representa un solo eslabón del robot y permite especificar sus propiedades, como tamaño, forma, color, o una malla 3D compleja importada (típicamente en formato .dae o .stl). Cada eslabón puede además contener propiedades dinámicas como su masa o su matriz de inercia, así como diversas propiedades de colisión (normalmente estructuras geométricas simples). La sintaxis de esta etiqueta es la siguiente:

link
<link name="nombre del eslabon"> <inertial>.....</inertial> <visual> .....</visual> <collision>.....</collision> </link>

La Figura 2 brinda una idea de lo que se entiende por sección visual y de colisión. La sección visual representa el eslabón real del robot y puede estar dado por una malla compleja con bastante detalle que visualmente hace que el modelo sea similar al robot real. La sección de colisión es el área alrededor de la parte visual, típicamente muy simplificada, que se utiliza para detectar colisiones entre eslabones.

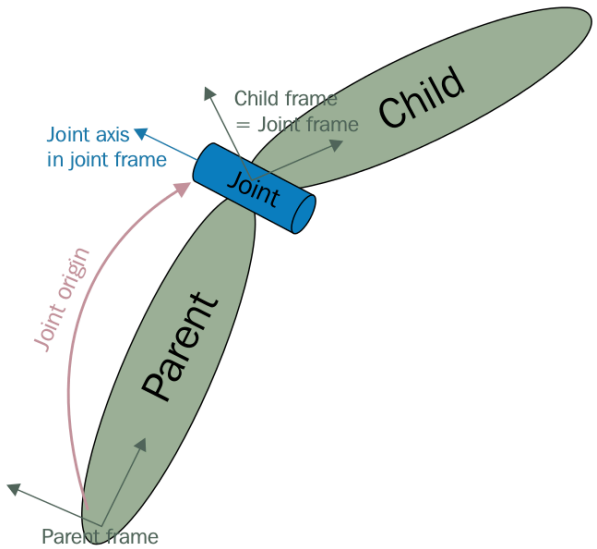

Figure 2: Representación de un eslabón en URDF

2. joint: Esta etiqueta representa cada una de las articulaciones del robot. Se puede especificar parámetros relacionados con la cinemática y la dinámica de cada articulación, así como establecer los límites del movimiento articular, de su velocidad, y del máximo esfuerzo mecánico que pueden aplicar (torque máximo). Esta etiqueta soporta articulaciones de revolución, contínuas, prismáticas, fijas y flotantes. **Las articulaciones fijas estrictamente no son articulaciones ya que no permiten el movimiento, pero se utilizan como un elemento que permite unir de manera fija dos eslabones entre sí**. Algunos parámetros de la sintaxis de una articulación se muestran a continuación:

joint

```
<joint name="nombre de la articulacion">
  <parent link="link1"/>
  <child link="link2"/>
  <calibration .... />
  <dynamics damping ..../>
  <limit effort .... />
</joint>
```

La Figura 3 muestra un esquema donde se puede ver lo que se conoce como el eslabón padre y el hijo. **Cada articulación une dos eslabones y queda completamente definida sólo cuando se especifica ambos eslabones que une: el primero es llamado el padre (parent) y el segundo el hijo (child).** Como se observa en la figura, el origen de la articulación se especifica con respecto al sistema de referencia del eslabón padre.

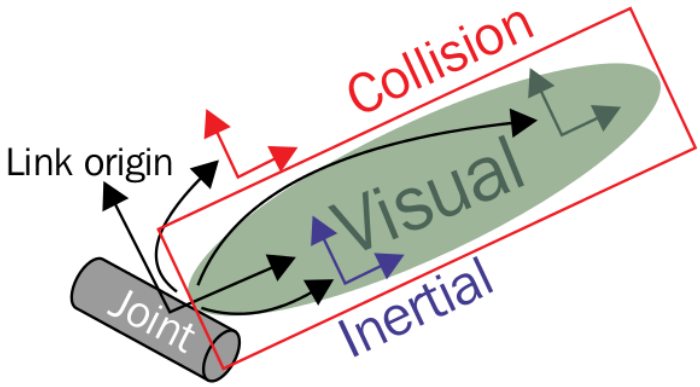

Figure 3: Representación de una articulación en URDF

3. robot: **Esta etiqueta encapsula todo el modelo del robot. Dentro de esta etiqueta se define el nombre del robot, todos sus eslabones y todas sus articulaciones.** La sintaxis es la siguiente:

robot
<robot name="<name of the robot>" <link> ..... </link> <link> ..... </link> <joint> ..... </joint> <joint> .....</joint> </robot>

donde pueden existir tantas etiquetas de tipo “link” y “joint” como sean necesarias. Un modelo de robot consiste en una serie de articulaciones y eslabones conectados. Un ejemplo se muestra en la Figura 4.

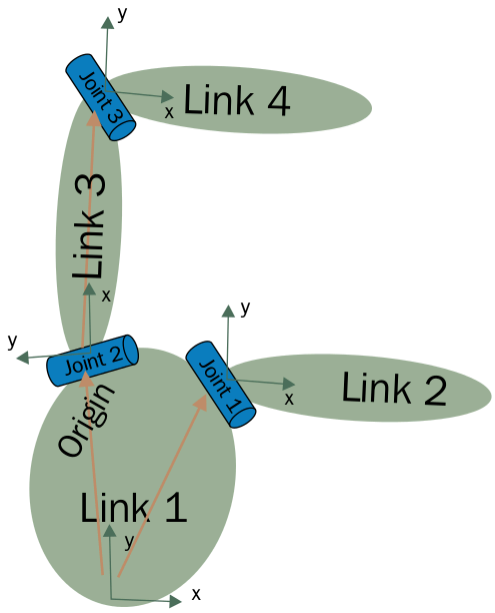

Figure 4: Ejemplo del modelo de un robot en URDF

4. gazebo: **Esta etiqueta se utiliza cuando se incluye parámetros de simulación para el simulador llamado Gazebo**, el cual es utilizado por defecto en ROS2 para simulaciones dinámicas. Se puede usar esta etiqueta para incluir “plugins” específicos de gazebo, material (color, textura) que se mostrará en el simulador, así como otras propiedades específicas.

Se puede encontrar mucha más información sobre etiquetas de URDF en el docs de URDF en ROS2. De igual manera, más información sobre las etiquetas de los eslabones está disponible en: https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Building-a-Visual-Robot-Model-with-URDF-from-Scratch.html

B - ALGUNOS ELEMENTOS DE GAZEBO

Para la simulación dinámica en Gazebo, es necesario añadir algunos elementos adicionales al URDF. Algunos de estos elementos son los siguientes.

Material: **El color en Gazebo no está relacionado con el color en RViz y debe ser especificado por etiquetas especiales de tipo "material"**. Por ejemplo, el siguiente código añade color blanco al eslabón de la base:

```
<gazebo reference="base_link">
  <material>Gazebo/White</material>
</gazebo>
```

ROS2 Control: **Para accionar el robot utilizando controladores de ROS2, en simulación, se debe definir el elemento llamado "ros2_control", el cual se encarga de establecer la relación de cada actuador con su respectiva articulación**, y así permite el movimiento de las articulaciones. Esto se realiza de la siguiente manera:

```
<ros2_control name="GazeboSystem" type="system">
  <hardware>
    <plugin>gazebo_ros2_control/GazeboSystem</plugin>
  </hardware>
  <joint name="pan_joint">
    <command_interface name="position">
      <param name="min">-3.1</param>
      <param name="max">3.1</param>
    </command_interface>
    <state_interface name="position"/>
  </joint>
  <joint name="tilt_joint">
    <command_interface name="position">
      <param name="min">-0.85</param>
      <param name="max">0.85</param>
    </command_interface>
    <state_interface name="position"/>
  </joint>
</ros2_control>
```

En esta etiqueta, joint name representa el nombre de la articulación a la cual se le asocia un actuador. La etiqueta "type" hace referencia al tipo de transmisión; en este caso se trata de una transmisión simple (SimpleTransmission). El elemento "hardwareInterface" es el tipo de interfaz de hardware que se utiliza en el robot y puede ser de posición, velocidad o fuerza (position,

velocity, effort). Esta interfaz de hardware es cargada desde el plugin de “gazebo_ros2_control”. Es posible además especificar la reducción mecánica, que en el ejemplo mostrado es igual a la unidad.

Plugin de Control: **El control se especifica mediante el plugin “gazebo_ros2_control”, el cual asigna las interfaces de hardware necesarias y el controlador.** El plugin se añade de la siguiente manera:

```
<gazebo>
  <plugin name="gazebo_ros2_control" filename="libgazebo_ros2_control.so">
    <parameters>$(find lab3)/config/pid_controllers.yaml</parameters>
  </plugin>
</gazebo>
```